Parul University

FACULTY OF ENGINEERING & TECHNOLOGY

BACHELOR OF TECHNOLOGY

INFORMATION & NETWORK SECURITY (INS)
Laboratory (303105376)

7th SEMESTER

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

LABORATORY MANUAL

Instructions to students

1. Every student should obtain a copy of laboratory Manual.
2. Dress Code: Students must come to the laboratory wearing.
   i. Trousers,
   ii. half-sleeve tops and
   iii. Leather shoes. Half pants, loosely hanging garments and slippers are not allowed.
3. To avoid injury, the student must take the permission of the laboratory staff before handling any machine.
4. Students must ensure that their work areas are clean and dry to avoid slipping.
5. Do not eat or drink in the laboratory.
6. Do not remove anything from the computer laboratory without permission.
7. Do not touch, connect or disconnect any plug or cable without your lecturer/laboratory technician's permission.
8. All students need to perform the practical/program.

CERTIFICATE

This is to certify that

Mr. /Ms……..

with enrolment no ..... has successfully completed his/her laboratory experiments in the ***Information and Network Security Laboratory (ML) (303105376)*** *from the department of …….. during the academic year .....*

योगः कर्मसु कौशलम्

PARUL UNIVERSITY

Date of Submission: ..... Staff In charge: .....

Head of Department: .....

INDEX

Sr. No.	Experiment Title	Page No.		Date of Performance	Date of Assessment	Marks out of 10	Sign
	Practical Set	**From**	**To**				
1	Implement Caesar Cipher Encryption-Decryption.						
2	Implement Monoalphabetic Cipher Encryption-Decryption.						
3	Implement Playfair Cipher Encryption-Decryption.						
4	Implement Polyalphabetic Cipher Encryption-Decryption.						
5	Implement Hill Cipher Encryption-Decryption.						
6	Implement Simple Transposition Encryption-Decryption.						
7	Implement One Time Pad Encryption-Decryption.						
8	Implement Diffie-Hellman Key Exchange Method.						
9	Implement RSA Encryption-Decryption Algorithm.						
10	Demonstrate Working of Digital Signature using Cryptool.						

Practical:-1

AIM: - Implement Caesar Cipher Encryption-Decryption.

THEORY: - The Caesar Cipher is a foundational cryptographic technique known as a substitution cipher, famously used by Julius Caesar to obscure military communications. It works by replacing each letter in the original message with a letter a fixed number of positions down the alphabet, seamlessly wrapping from 'Z' back to 'A'. Mathematically, this substitution is represented by the formula E(x) = (x + n) mod {26}, where 'x' represents the letter's numerical index (0–25) and 'n' is the chosen shift value. While elegantly simple and a great introduction to encryption concepts, it provides practically zero security in the modern era; an attacker can instantly crack it by attempting all 25 possible shifts (a brute-force attack) or by analyzing the frequency of the letters, which the cipher completely fails to mask.

CODE:–

```
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

// The core Caesar Cipher logic
string caesarCipher(const string& text, int shift, bool encrypt = true) {
    string result = "";

    // Reverse shift for decryption
    if (!encrypt) {
        shift = -shift;
    }

    for (char c : text) {
        if (isupper(c)) {
            result += char(int(c - 'A' + shift % 26 + 26) % 26 + 'A');
        }
        else if (islower(c)) {
            result += char(int(c - 'a' + shift % 26 + 26) % 26 + 'a');
        }
        else {
            result += c;
        }
    }
    return result;
}

int main() {
    int option;
    string text;
    int key;

    // Print the menu exactly as shown in the image
    cout << "Choose an option:\n";
    cout << "1. Encryption\n";
```

```
    cout << "2. Decryption\n";
    cin >> option;

    if (option == 1) {
        cout << "Enter the plaintext: ";
        cin >> ws; // Clears any trailing newlines from previous inputs
        getline(cin, text); // Allows for text with spaces

        cout << "Enter the key (integer shift value): ";
        cin >> key;

        cout << "Encrypted text: " << caesarCipher(text, key, true) << endl;
    }
    else if (option == 2) {
        cout << "Enter the text: ";
        cin >> ws;
        getline(cin, text);

        cout << "Enter the key (integer shift value): ";
        cin >> key;

        cout << "Decrypted text: " << caesarCipher(text, key, false) << endl;
    }
    else {
        cout << "Invalid option selected." << endl;
    }

    return 0;
}
```

OUTPUT:-

1) Encryption:

```
Choose an option:
1. Encryption
2. Decryption
1
Enter the plaintext: Dhruv
Enter the key (integer shift value): 2
Encrypted text: Fjtwx
```

2) Decryption:

```
Choose an option:
1. Encryption
2. Decryption
2
Enter the text: fjtwx
Enter the key (integer shift value): 2
Decrypted text: dhruv
```

Practical:-2

AIM:- Implement Monoalphabetic Cipher Encryption-Decryption.

THEORY:- A **Monoalphabetic Cipher** relies on a fixed 1-to-1 mapping where each letter of the standard alphabet is replaced by a unique letter from a randomly jumbled 26-letter key. Unlike the Caesar cipher, which only offers 25 possible shifts, the monoalphabetic cipher offers 26 (over 400 septillion) possible keys, making a brute-force attack computationally impossible. However, despite this massive key space, it provides very weak security in practice. Because the cipher is strictly one-to-one, it preserves the natural letter patterns of the plaintext. An attacker can easily crack it using **frequency analysis** by matching the most common letters in the cipher text to the most common letters in the underlying language (like 'E', 'T', and 'A' in English).

CODE:–

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

string plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
string cipher = "QWERTYUIOPASDFGHJKLZXCVBNM";

// Encryption Function
string encrypt(string text)
{
   string result = "";

   for (char ch : text)
   {
      if (isalpha(ch))
      {
         bool lower = islower(ch);
         ch = toupper(ch);

         int index = plain.find(ch);
         char enc = cipher[index];

         if (lower)
            enc = tolower(enc);

         result += enc;
      }
      else
      {
         result += ch;
      }
   }
```

```
    return result;
}

// Decryption Function
string decrypt(string text)
{
    string result = "";

    for (char ch : text)
    {
        if (isalpha(ch))
        {
            bool lower = islower(ch);
            ch = toupper(ch);

            int index = cipher.find(ch);
            char dec = plain[index];

            if (lower)
                dec = tolower(dec);

            result += dec;
        }
        else
        {
            result += ch;
        }
    }

    return result;
}

int main()
{
    int choice;
    string text;

    cout << "===== Monoalphabetic Cipher =====\n";
    cout << "1. Encryption\n";
    cout << "2. Decryption\n";
    cout << "Enter your choice: ";
    cin >> choice;
    cin.ignore();
```

```
    if (choice == 1)
    {
        cout << "\nEnter Plain Text: ";
        getline(cin, text);

        cout << "Encrypted Text: " << encrypt(text) << endl;
    }
    else if (choice == 2)
    {
        cout << "\nEnter Cipher Text: ";
        getline(cin, text);

        cout << "Decrypted Text: " << decrypt(text) << endl;
    }
    else
    {
        cout << "\nInvalid Choice!" << endl;
    }

    return 0;
}
```

OUTPUT:-

1) Encryption:

```
===== Monoalphabetic Cipher =====
1. Encryption
2. Decryption
Enter your choice: 1

Enter Plain Text: Dhruv
Encrypted Text: Rikxc
```

2) Decryption:

```
===== Monoalphabetic Cipher =====
1. Encryption
2. Decryption
Enter your choice: 2

Enter Cipher Text: rikxc
Decrypted Text: dhruv
```